

Abstract Data Types

L32 - Introduction to Structures

Department of Computer Science
University of Pretoria

03 June 2024

Abstract Data Types:

An abstract data type is a data type that specifies:

- 1 values that can be stored in a variable of this type
- 2 Operations that can be performed on the values

Users of an abstract data type (i.e. other programmers) do not need to know the implementation of the data type i.e. how the data is stored and how the operations are implemented.

- 2nd aggregate data type: **struct**.
- **Recall:** aggregate meaning "grouping".
- **Recall array:** collection of values of same type.
- **Structure:** collection of values of different types
- Treated as a single item, like arrays.
- **Major difference:** Must first "define" struct prior to declaring any variables.

C/C++ construct that allows multiple variables to be grouped together.

General Format:

```
1 struct <structName>
2 {
3     type1        field1 ;
4     type2        field2 ;
5     . . .
6 } ;
```

Example of a struct Declaration:

```
struct Student //Structure name begins with uppercase letter
{
    string firstName; //structure fields
    string surname;
    string course;
    int mark;
} ; //compulsory semi-colon
```

Multiple fields of same type can be in comma-separated list, e.g. string firstName, surname, course;

Defining Variables

struct declaration does not allocate memory or create variables. It is just a type definition

```
struct Student    //Structure name begins with uppercase letter  
{  
    string firstName; //structure fields  
    string surname;  
    string course;  
    int mark;  
} ;    //compulsory semi-colon
```

- To define a struct variable in C++: e.g. *Student goodStudent;*
- or C-style declaration *struct Student goodStudent;*

Accessing Structure Members (fields)

Use the **dot (.) operator** to refer to member variables (fields) of struct variables, e.g.

```
cin >> goodStudent.firstName; //or use getline
cin >> goodStudent.surname; //or use getline
cin >> goodStudent.course; //or use getline
cin >> goodStudent.marks;
//Member variables (fields) can be used in any manner appropriate
//for their data type as shown below.
cout << goodStudent.firstName;
cout << goodStudent.surname;
```

Struct full code example:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 //type definitions
5 struct Student {
6     string firstName;
7     string surname;
8     string course;
9     int marks;
10 };
11
12 int main() {
13     Student goodStudent;
14     //read student data
15     cout << "Enter-student-details:" << endl;
16     cout << "First-name:-";
17     cin >> goodStudent.firstName;
18
19     cout << "Surname:-";
20     cin >> goodStudent.surname;
```


Struct full code example (continued):

```
1      cout << " Course-code:-" ;
2      cin >> goodStudent.course ;
3      cout << " Marks-for-course:-" ;
4      cin >> goodStudent.marks ;
5      //display student data
6      cout << endl << endl ;
7      cout << " Student-details-are:"  << endl ;
8      cout << " First-name:-" <<
9      goodStudent.firstName << endl ;
10     cout << " Surname:-" <<
11     goodStudent.surname << endl ;
12     cout << " Course-code:-" <<
13     goodStudent.course << endl ;
14     cout << " Marks-for-course:-" << goodStudent.marks << endl ;
15     return 0 ;
16 }
```

Class Exercise: Given the following code snippet:

```
1 #include <iostream>
2 using namespace std;
3 struct Rectangle {
4     // Data members
5     double length;
6     double width;
7 };
8 // Function to calculate the area
9     double calculateArea(double length, double width) {
10         return length * width;
11     }
12 int main() {
13     // write your code here
14     return 0;
15 }
```

Complete the code that it calculates the area of the rectangle from user input. Make

sure you use of the provided struct as well as the function. 

Class Exercise Solution:

```
1  int main() {
2      // Create an instance of Rectangle
3      Rectangle rect;
4
5      // Get values from user
6      cout<<"Enter-Rectangle-length:-";
7      cin>> rect.length;
8      cout<<"Enter-Rectangle-width:-";
9      cin>> rect.width;
10
11     // Call the function to calculate the area
12     double area = calculateArea(rect.length, rect.width);
13
14     // Print the area
15     cout << "The-area-of-the-rectangle-is:-" << area << endl;
16     return 0;
17 }
```

Comparing struct Variables

- Given the declarations:

Rectangle rect1, rect2;

- Cannot compare struct variables directly:

if (rect1 == rect2) // won't work

- Instead, must compare on a field basis:

```
if ((rect1.length == rect2.length) &&  
    (rect1.width == rect2.width))  
{  
    .....  
}
```

Initialising Structures

Can initialize at declaration e.g:

```
struct Date    {  
    int month;  
    int day;  
    int year;  
};
```

```
Date dueDate = {12, 31, 2003};
```

Declaration provides initial data to all three member variables.

Structure Assignments

Given structure named **CropYield**

Declare two structure variables:

```
CropYield apples, oranges;
```

Both are variables of "struct type CropYield"

- Simple assignments are legal: *apples = oranges;*
- Simply copies each member variable from apples into member variables from oranges.

Values can be assigned to a struct variable e.g. type definition:

```
struct Student
{
    string studentNum;
    string firstName;
    string surname;
    int marks;
};
```

```
//e.g variable declaration:
Student oneStudent;
```

```
//e.g. assign values to fields (members) of the struct variable:
oneStudent.studentNum = "1234";
oneStudent.firstName = "Mpho";
```

Structures as function arguments:

Passed like any simple data type:

- Pass-by-value
- Pass-by-reference
- Or a combination

Can also be returned by function:

- Return-type is structure type

Pass-by-Value

In pass-by-value, a copy of the structure is passed to the function. Changes made to the structure inside the function **do not** affect the original structure.

```
1 #include <iostream>
2 using namespace std;
3 // Defining a struct called Point
4 struct Point {
5     int x;
6     int y;
7 };
8 // Function to display the point (pass-by-value)
9 void displayPoint(Point p) {
10     cout << "Point-(" << p.x << " ,-" << p.y << ")" << endl;
11 }
12 int main() {
13     // Creating an instance of Point
14     Point pt = {10, 20};
15     // Calling the function with the Point instance
16     displayPoint(pt);
17     return 0;
18 }
```

Pass-by-Reference

In pass-by-reference, the function receives a reference to the structure. Changes made to the structure inside the function **affect** the original structure.

```
1 #include <iostream>
2 using namespace std;
3 struct Point {
4     int x;
5     int y;
6 };
7 // Function to update the point (pass-by-reference)
8 void updatePoint(Point &p) {
9     p.x += 10;
10    p.y += 20;
11 }
12 int main() {
13     Point pt = {10, 20};
14     // Calling the function with the Point instance
15     updatePoint(pt);
16     // Displaying the updated point
17     cout << "Updated-Point-(" << pt.x << " ,-" << pt.y << ")";
18     return 0;
19 }
```

Combination of Pass-by-Value and Pass-by-Reference (1)

In the example below, one function demonstrates pass-by-value while another demonstrates pass-by-reference.

```
1 #include <iostream>
2 using namespace std;
3 // Define a struct called Rectangle
4 struct Rectangle {
5     double length;
6     double width;
7 };
8 //Function to calculate area (pass-by-value)
9 double calculateArea(Rectangle rect) {
10     return rect.length * rect.width;
11 }
12 //Function to double the size of the rectangle (pass-by-reference)
13 void doubleSize(Rectangle &rect) {
14     rect.length *= 2;
15     rect.width *= 2;
16 }
```

Combination of Pass-by-Value and Pass-by-Reference (2)

In the example below, one function demonstrates pass-by-value while another demonstrates pass-by-reference.

```
1  int main() {  
2      // Create an instance of Rectangle  
3      Rectangle rect = {5.0, 3.0};  
4  
5      // Calculate and display the area (pass-by-value)  
6      double area = calculateArea(rect);  
7      cout << "Original-area:-" << area << endl;  
8  
9      // Double the size of the rectangle (pass-by-reference)  
10     doubleSize(rect);  
11  
12     // Calculate and display the new area  
13     area = calculateArea(rect);  
14     cout << "New-area-after-doubling-size:-" << area << endl;  
15     return 0;  
16 }
```

Return Type is Structure Type

In the example below, the return type of the function is a structure type. The function creates and returns a structure.

```
1 #include <iostream>
2 using namespace std;
3 struct Point {
4     int x, y;
5 };
6 // Function that returns a Point
7 Point createPoint(int x, int y) {
8     Point p;
9     p.x = x;
10    p.y = y;
11    return p; // Return the structure
12 }
13 int main() {
14     // Call the function and get the returned structure
15     Point pt = createPoint(10, 20);
16     // Display the point
17     cout << "Point-(" << pt.x << ", -" << pt.y << ")" << endl;
18     return 0;
19 }
```